

Lipschitz Quaternion Ends — Module Specification

Author: Generated for the user by ChatGPT

Lipschitz Quaternion Ends — Module Specification

Author: Generated for the user by ChatGPT

Overview

=====

This document describes a developer specification for a parallel module to your existing Prime-Ends Topology module.

The new module (the "Lipschitz Quaternion Ends" module) produces motion-aware maps and discrete motion anchors

by encoding local image/video patch features as quaternion-like 4D vectors, quantizing them to a Lipschitz integer lattice,

and running a variant of the existing prime-ends construction over those seeds.

Goals

=====

- Produce a compact, persistent set of motion anchors ("Lipschitz point-ends") suitable for tracking, motion-driven generation, and fusion with topology.
- Reuse the same base pixel-graph and prime-ends algorithm so outputs are comparable.
- Provide clear APIs and data structures so developers can implement, test, and integrate the module.

Key concepts

=====

- Pixel Graph (G): The common adjacency graph used by both the prime-ends and Lipschitz modules. MUST be identical between the two modules for comparability.
- Quaternion Field (Q): A per-pixel 4-vector $q = (w, x, y, z)$ encoding motion/orientation/feature information.
- Lipschitz Lattice Quantization: Mapping Q to integer 4-tuples (a,b,c,d) by rounding / scaling.
- Seeds (S_L): Sparse set of pixels chosen by properties of quantized quaternions (norm thresholding, small-norm selection, morphological filtering).
- Lipschitz Point-Ends: Equivalence classes of approach sequences to the boundary computed from the same cross-cut / chain algorithm used for Gaussian prime-ends but seeded by S_L.
- Motion Descriptors: Per-end descriptors (avg quaternion, centroid velocity, curl/divergence stats, lifetime).

Design overview

=====

1. Input:

- Single frame I_t ($H \times W \times 3$), and optionally previous frame I_{t-1} , depth D_t .
- Base pixel graph G (constructed once per frame, same adjacency as prime-ends module).

2. Core steps:

- Compute dense optical flow (or short-term motion estimate) between I_{t-1} and I_t .
- Compute local features: flow components, gradient magnitude, Laplacian, depth derivatives, curl/divergence as needed.
- Assemble quaternion field $Q(x) = (w, x, y, z)$ using a chosen mapping. Normalize if required.
- Quantize Q to integer Lipschitz lattice Q_{int} by scaling and rounding.
- Select seeds S_L by norm thresholds, frequency-of-occurrence, and morphological filtering.
- Use the Prime-Ends "cross-cut and chain" algorithm on the base graph G with seeds S_L to produce Lipschitz point-ends.
- For each end compute per-end descriptors, masks, and persistence statistics.
- (Optional) Match Lipschitz ends to prime-ends (IoU + Hungarian) to fuse motion and topology.

3. Outputs:

- Per-pixel quaternion field (float32 $H \times W \times 4$).
- Quantized lattice map (int32 $H \times W \times 4$).

- Seed mask S_L (boolean HxW).
- Lipschitz ends: list of End objects each with:
 - * mask (HxW boolean)
 - * seed_mask (subset of S_L)
 - * centroid (x,y)
 - * avg_quaternion (w,x,y,z)
 - * motion_stats (mean_speed, var_speed, curl, divergence)
 - * lifetime (frames)
 - * topo_invariants (local b0,b1 or persistence)
- Motion maps: motion magnitude map, rotational_map, motion_ends_map, saliency_map
- Match results vs prime-ends: IoU matrix, matches, unmatched lists.

APIs and data structures

=====

Design the API so it fits into your existing codebase. The following is a recommended Python-style interface.

Classes and types

- PixelGraph
 - attributes: height, width, adjacency (sparse)
 - factory: from_image_dimensions(height, width, adjacency_rule='4-neigh'|'8-neigh')
- QuaternionField
 - numpy array shape (H, W, 4) float32
 - methods: normalize(), to_lipschitz(scale) -> LipschitzLatticeMap
- LipschitzLatticeMap
 - numpy array shape (H, W, 4) int32
 - attributes: norms (H, W), seed_mask (H, W boolean)
- End (dataclass)
 - mask: numpy boolean HxW
 - seed_mask: numpy boolean HxW
 - centroid: tuple (x,y)
 - avg_quaternion: 4-vector float
 - motion_stats: dict
 - topo_invariants: dict

Module functions

- compute_quaternion_field(frame_t, frame_tm1=None, depth_t=None, config) -> QuaternionField
- quantize_lipschitz(quaternion_field, scale=0.2, norm_thresh=(0,5)) -> LipschitzLatticeMap
- select_seeds(lattice_map, min_size_px=16, connectivity=8) -> seed_mask (HxW)
- compute_point_ends(pixel_graph: PixelGraph, seed_mask: np.ndarray) -> List[End]
- (This should reuse your existing prime-ends code but accept arbitrary seeds.)
- compute_motion_descriptors(ends, quaternion_field, flow_field) -> annotated ends
- match_ends(prime_ends, lipschitz_ends) -> matches, IoU_matrix
- generate_motion_maps(quaternion_field, ends, annotated_ends) -> dict of images/maps

Configuration options

=====

- adjacency_rule: '4' or '8' neighbor graph
- flow_method: 'farneback' | 'tv-l1' | 'raft' (RAFT is recommended for accuracy)
- q_mapping: mapping selection among predefined choices (see "Quaternion mappings" below)
- quantization_scale: float controlling lattice coarseness
- norm_thresholds: (min, max) for seed selection
- morphological_filters: kernel sizes, min_component_size
- temporal_smoothing_alpha: EMA coefficient for per-seed quaternion averaging

- iou_match_threshold: threshold above which a match is considered valid

Quaternion mappings (recommended options)

=====

The quaternion field is central — several sensible mappings:

1. Flow + gradient (simple):

- $w = \tanh(\alpha * ||\text{flow}||)$
- $x = \text{flow}_x$
- $y = \text{flow}_y$
- $z = \text{gradient_magnitude (Sobel)}$
- normalize optionally

2. Flow + curl/divergence:

- $w = \text{flow_magnitude}$
- $x = \text{curl}$
- $y = \text{divergence}$
- $z = \text{laplacian or edge strength}$

3. Depth-aware rotation (if depth available):

- compute local 3D rigid transform using KLT + depth within a patch
- convert 3D rotation to unit quaternion (w, x, y, z)

4. Multi-channel fused:

- fold color, depth, and flow into a 4-vector using PCA or learned projection, then map to quaternion.

Quantization and seed selection

=====

- Scale each quaternion component to comparable numeric ranges (z-score or tanh) before quantization.
- Quantize: $Q_int = \text{round}(Q / \text{scale})$
- Norm-based selection: $\text{norms} = \sqrt{a^2 + b^2 + c^2 + d^2}$
- Keep $0 < \text{norms} \leq R$ (small-norm points often correspond to subtle motion)
- Alternatively keep $\text{norms} \geq R$ if targeting strong-motion anchors
- Frequency and stability filter (temporal):
- Keep only pixels that map to the same lattice label for k consecutive frames (or exponential smoothing of labels)
- Morphological filtering:
- Remove seed components smaller than min_size_px
- Optionally dilate seeds to ensure the prime-ends code has enough support

Reusing prime-ends code

=====

- The prime-ends code expects a graph G and a seed set S (e.g., Gaussian primes were seeds).
- Ensure you extract and provide the seed set in identical format (boolean mask).
- Run `compute_point_ends(G, S_L)` to get Lipschitz ends.
- Because the pixel graph is unchanged between runs, topological differences arise only from seed selection.

Fusion: matching motion $\leftrightarrow$ topology

=====

- For comparison and fusion:
- 1. For each end produce an accessible-boundary mask (binary).
- 2. Compute pairwise IoU between prime-end masks and Lipschitz-end masks.
- 3. Use Hungarian algorithm (maximize IoU) for one-to-one matching; keep unmatched ends separate.
- 4. For matched pairs, attach motion descriptors to prime-ends. For unmatched motion ends, mark as motion-only anchors.

Motion maps and downstream uses

=====

- Motion Map types:

- * motion_magnitude_map: per-pixel $\|flow\|$
- * rotational_map: curl magnitude or quaternion orientation visualized in HSV
- * motion_ends_map: per-pixel end ID label for Lipschitz ends
- * saliency_map: combination of motion magnitude and membership to Lipschitz seeds

- Uses:

- * Drive particle systems anchored at end centroids with motion descriptors.
- * Guide deformation fields or texture warps along quaternion orientations.
- * Improve tracking and segmentation by combining topological anchors (prime-ends) with motion anchors (Lipschitz ends).
- * Provide compact motion controls to generative models (per-end quaternion + centroid + lifetime).

Evaluation & metrics

=====

- Spatial similarity:

- * mean IoU of matched ends, distribution of IoU scores

- Topological similarity:

- * b0/b1 differences, bottleneck distance between persistence diagrams computed on matched regions

- Temporal coherence:

- * fraction of Lipschitz ends persisting for $\geq k$ frames

- Motion fidelity:

- * MSE / cosine similarity vs measured optical flow or ground truth flows if available

- Ablation studies:

- * Vary quantization_scale, seed thresholds, quaternion mapping, and measure metric sensitivity

Implementation notes & libraries

=====

Recommended Python libraries:

- OpenCV (cv2) — optical flow, image I/O
- RAFT implementation (PyTorch) — if you need accurate flow
- numpy, scipy — numerical work, linear assignment
- networkx or custom graph representation — pixel graph
- ripser / gudhi — persistent homology
- numpy-quaternion or custom quaternion utils — quaternion averaging
- scikit-image — morphological operations, labeling
- matplotlib for debugging visualizations

Performance considerations

=====

- Downscale frames for heavy computations (e.g., max side = 512 px).
- Precompute and reuse pixel graph G across frames of same size.
- Use RAFT on a downsampled scale or every nth frame if compute-limited.
- Seed sparsity is a major performance lever: coarser quantization $\rightarrow$ fewer seeds $\rightarrow$ faster end computation.

Testing & validation plan

=====

1. Unit tests:

- quaternion field outputs expected shapes/ranges on synthetic inputs
- quantization deterministic (given seed)
- seed selection respects thresholds and morphology
- compute_point_ends runs and returns reasonable end masks on small synthetic grids

2. Integration tests:

- run full pipeline on synthetic videos (rotating disk, translating box)
- measure IoU vs baseline motion segmentation

3. Regression & stability:

- temporal persistence test: track seeds and ends across frames; report lifespan distributions

4. Visual tests:

- overlay seeds, ends, and motion maps on frames and manually inspect

Developer pseudocode & example

=====

Below is a concise code sketch; this should slot into your project and call into existing prime-ends functions.

```
# compute quaternion field
Q = compute_quaternion_field(frame_t, frame_tm1, depth_t, config)
# quantize to lipschitz lattice
Q_int, seeds = quantize_lipschitz(Q, scale=config.scale, norm_thresh=config.norm_thresh)
# morphological filtering
seeds = morphological_filter(seeds, min_size=config.min_seed_size)
# compute lipschitz ends using existing prime-ends code
ends_L = compute_point_ends(G, seeds)
# compute descriptors
ends_L = compute_motion_descriptors(ends_L, Q, flow_field)
# match to prime-ends
matches, IoU = match_ends(prime_ends, ends_L)
# generate motion maps for downstream use
maps = generate_motion_maps(Q, ends_L, config)
```

Integration checklist

=====

- [] Ensure pixel graph G is the same for both modules.
- [] Provide well-documented config parameters and sensible defaults.
- [] Implement stable quaternion averaging.
- [] Provide visualization tool for tuning quantization_scale & thresholds.
- [] Provide unit/integration tests and a short sample video for demo.

Potential extensions

=====

- Learned quantization: train a small network to map per-pixel features to lattice indices that maximize temporal stability.
- Multi-scale seeds: maintain seeds at multiple quantization scales and fuse ends across scales.
- Use Lipschitz ends as supervision signals for training motion segmentation networks.

Appendix: Parameter defaults (starting point)

=====

- adjacency_rule: 8-neighbor
- flow_method: 'farneback' (prototype), 'raft' (production)
- q_mapping: flow + gradient (Option 1)
- quantization_scale: 0.2
- norm_thresholds: (0.5, 6.0)
- min_seed_size: 16 pixels
- temporal_smoothing_alpha: 0.6
- iou_match_threshold: 0.4

Appendix: Quick references & further reading

=====

- Quaternion basics and averaging techniques (Shoemake/Markley)
- Carathéodory prime-ends and discrete analogues
- Persistent homology tutorials (Ripser, GUDHI)
- RAFT optical flow paper & implementation